Pattern Recognition 176 (2026) 113211



Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/pr





UPGP:Backdoor defense via unlearning perturbation and orthogonality-constraint gradient projection

Jingtai Li [a], Xiujiu Yuan [a,*], Jiwei Tian [b], Shiwei Lu [a], Dengxiu Yu [c,d]

[a] Fundamentals Department, Air Force Engineering University, No.1 A, Changle East Road, Xi'an, 710043, Shaanxi Province, China
[b] Ministry of Education Key Laboratory for Intelligent Networks and Network Security, School of Cyber Science and Engineering, Xi'an Jiaotong University, No. 28 Xianning West Road, Xi'an, 710049, Shaanxi Province, China
[c] the School of Artificial Intelligence, Optics and Electronics, Northwestern Polytechnical University, No.127, West Youyi Road, Xi'an, 710072, Shaanxi Province, China
[d] Key Laboratory of Cyberspace Security, Ministry of Education, China, No.62, Science Road, Zhongyuan Distric, Zhengzhou, 450001, Henan Province, China

ARTICLE INFO

Keywords:
Data security
Backdoor defense
Machine unlearning
Learning dynamic
Gradient projection

ABSTRACT

In the field of artificial intelligence, the wide adoption of third-party data has heightened the risk of backdoor attacks based on data poisoning. Although post-training defenses can mitigate such attacks, aggressive strategies often degrade the model's performance on its main task. To address this, a novel method that combines backdoor detection and elimination through machine unlearning is proposed. Specifically, unlearning perturbation is first defined to capture the parameter variation induced by forgetting a subset of samples. Subsequently, experiments confirm that backdoor samples exhibit lower sensitivity to perturbations generated from normal samples. In addition, a learning-dynamics analysis attributes this discrepancy to unlearning sensitivity, which is defined as the inner product between the gradients of normal and backdoor samples. This analysis further demonstrates that this metric quantifies the extent to which backdoor removal perturbs the model's main task. Leveraging this insight, an orthogonality-constrained gradient projection method projects the unlearning gradient onto the null space of the normal-sample gradient, thereby eliminating the aforementioned unlearning sensitivity and preserving the accuracy of normal samples. The proposed method is evaluated across six backdoor attack scenarios and two network architectures, reducing the average attack success rate by 96.34 percentage points and improving robust accuracy by 83.68 percentage points, while maintaining the model's performance on the main task.

1. Introduction

With the development of artificial intelligence technology, the scale of training dataset is also becoming increasingly large, making it easier for attackers to carry out backdoor attacks on machine learning models through data poisoning. A model trained on poisoned dataset will be implanted with a backdoor, causing the model to generate abnormal outputs when processing trigger-containing samples [1]. However, relying solely on backdoor sample detection before training is insufficient to completely defend against the backdoor attacks [2], and retraining the attacked model incurs extremely high costs. It is necessary to devise post-training defense methods to restore the system to its normal operational state after a data poisoning attack, thereby fulfilling the maintainability requirements of artificial intelligence services.

In the field of backdoor defense of convolutional neural networks for image classification, post-training defense methods have attracted widespread attention [3]. In the post-training defense scenario, information about backdoor samples is obtained by leveraging the backdoor model, and the model is purified by eliminating the backdoor. Although existing post-training defense methods can effectively eliminate backdoor in models, there is still a need to further address how to reduce the impact of backdoor elimination on the model's main task. To this end, existing methods mainly include detecting neurons related to the backdoor [4], suppressing or modifying the backdoor features in the feature map [5], or using normal samples for restoration. However, these methods often require inserting extra model components or retraining with large volumes of clean data, which disrupts the original architecture and leads to substantial computational overhead as well as considerable implementation complexity.

To reduce the impact of backdoor elimination on the model's main task, we propose a novel post-training backdoor defense approach based on unlearning perturbation and gradient projection (UPGP), without the need for extra feature extraction or dimensionality reduction methods. Since controllable and predictable model perturbations enable the

* Corresponding author.
E-mail addresses: jingtai_li@163.com (J. Li), xiujiuyuan@163.com (X. Yuan), tianjiwei2016@163.com (J. Tian), 3130104597@zju.edu.cn (S. Lu), yudengxiu@nwpu.edu.cn (D. Yu).

https://doi.org/10.1016/j.patcog.2026.113211
Received 2 September 2025; Received in revised form 1 January 2026; Accepted 29 January 2026
Available online 31 January 2026
0031-3203/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

detection of model anomalies, the concept of unlearning perturbation was introduced to analyze the unlearning impact on model evaluation metrics of other samples. However, the model's logits output of backdoor samples exhibit greater resistance to the unlearning perturbation of normal samples within the same category. According to this phenomenon, the backdoor model can be used to detect the target categories of backdoor attacks and screen out a portion of backdoor samples with high confidence level for backdoor unlearning in the next step.

In addition, the analysis of learning dynamics shown that the inner product of the gradient of cross-entropy loss function of the normal samples and the gradient of unlearning loss function of the backdoor samples determines the unlearning sensitivity of normal samples to forgotten backdoor samples. Accordingly, the unlearning gradient is projected onto the null space of the gradient matrix formed by normal samples, so that the resulting orthogonality drives the unlearning sensitivity to zero and preserves the model's performance on normal data. Unlike indirect input-space method that merely suppress activation [6], our gradient-space projection constrains parameter updates. By projecting the unlearning gradient onto the sub-space orthogonal to the main-task gradient, we fundamentally eliminate interference with the normal samples. In addition, the projection is performed in a layer-wise manner, providing fine-grained protection for each layer and yielding superior preservation of model performance. Overall, the contributions of this paper are as follows:

(1) Unlearning perturbation and unlearning impact are introduced to formalize the analysis of model behavior under controlled perturbations. Using these concepts, backdoor samples are found to exhibit greater resistance to unlearning perturbations derived from normal samples within the same category. Based on this, a backdoor detection method is proposed to identify backdoor attack categories and backdoor samples. This work represents the first application of machine unlearning to anomaly detection and achieves identification without requiring additional feature extraction or dimensionality reduction models.

(2) The notion of unlearning sensitivity is proposed to quantify how removing specific samples impacts the model's main task. Analysis based on learning dynamics shows that this sensitivity equals the gradient inner product between normal and backdoor samples, which explains their disparate responses to unlearning perturbations. By minimizing this inner product, the unlearning sensitivity of normal samples to forgetting backdoor samples is reduced, thus mitigating the impact of backdoor unlearning on the main task.

(3) Guided by the concept of unlearning sensitivity, an orthogonality-constrained gradient projection method is proposed. Unlike vector orthogonal projection, the method constructs a gradient space for the parameters of each layer, forcing the unlearning gradient to be orthogonal to the main-task gradient space. Thus, the impact of backdoor unlearning on main-task performance is minimized. Experiments covering six backdoor attacks and six defense baselines demonstrate that our method reduces the average attack success rate by 96.34 percentage points (pp) and improves the robust accuracy by 83.68 pp, while maintaining the main-task accuracy.

The remainder of this article is organized as follows. Section 2 reviews the related work on post-training backdoor defense and machine unlearning. The proposed UPGP methods are detailed in Section 3. Section 4 presents the experimental results, including comparative experiments and analysis. Section 5 summarizes our work and introduces future research directions.

2. Related works

2.1. Post-training backdoor defense

First, it is necessary to obtain relevant information about the backdoor, there are three main approaches for achieving backdoor sample detection: (1) Based on the distribution differences between the backdoor samples and the normal samples in the latent feature space. Statistical contamination analyser method (SCAn) [7] relies on the expectation maximization (EM) algorithm to decompose the representation of samples. Hu and Chang [8] use a diffusion model to learn the feature space distribution of normal samples to identify backdoor samples. (2) Based on the strong correlation between the backdoor trigger and the backdoor target category. Liu et al. [9] take advantage of the fact that backdoor images and normal images exhibit distinct levels of robustness to image corruption. Wang et al. [3] utilize the difference in the maximum decision boundaries between backdoor samples and normal samples as a feature. Chen et al. [10] analyze backdoor samples by introducing a weaker backdoor. (3) Reconstruct model or use an external feature extractor. Wang et al. [11] remove mislabeled samples as backdoor samples by using self-supervised representation learning to represent the semantic information in the images. Li et al. [12] leverage a reputable feature extractor and compare the similarity of extracted feature vectors. However, the clustering-based backdoor sample detection method needs to construct a low-dimensional feature space due to the distance metric fails in the high-dimensional feature space and this process require additional computational and storage resources.

After obtaining the prior knowledge of backdoor samples, researchers eliminate model backdoor by detecting and pruning neurons that are related to backdoor samples [4], or use knowledge distillation techniques to extract harmless information from the backdoor model [13]. In addition, contrastive learning methods [14] and machine unlearning methods [15] are also used to eliminate the model's response to backdoor triggers. We need to focus on the issue of performance degradation of the model after eliminating the backdoor, such as the recognition ability for normal samples and the robustness against backdoor samples.

2.2. Machine unlearning

Machine unlearning is a technique aimed at eliminating the influence of marked training samples on a machine learning algorithm while preserving its utility. This approach enhances the protection of data privacy in the training dataset and enables the control of the model's behavior, thereby having a wide range of application scenarios. For example, it's used to mitigate the impact of backdoor samples in backdoor attack defense [15], resist model extraction attack [16], and suppress inappropriate elements in generative fields [17].

Retraining the model on a dataset with marked samples have been removed is a native method, but it is costly. Researchers have proposed several approaches to implement machine unlearning in deep learning models: (1) sample modification-based methods [18], which alter the samples in the dataset and train the model to interfere with its predictions; (2) dataset structure modification-based methods [19], which change the structure of the dataset and only retrain the portion containing the unlearning samples to reduce the cost of retraining; (3) directly obtain the model parameters [20], which modify the model parameters directly based on information from the unlearning samples; and (4) iteratively modify the model parameters [21], which fine-tune the model parameters using multiple learning paradigms or adjusting the loss function to achieve machine unlearning. When the defender can obtain the backdoor samples, the machine unlearning task highly coincides with the backdoor elimination task.

3. UPGP backdoor defense method

Anomalous model behavior can be detected through controlled perturbation, such as machine unlearning. Hence, we propose the concepts of unlearning perturbation and analyze unlearning impact to backdoor samples and normal samples. Table 1 provides an explanation of frequently used symbols throughout this paper.

3.1. Unlearning samples via influence function

The influence function is used to measure the importance of samples, and it can also be utilized for machine unlearning [22,23]. When a model M is trained on the dataset D_{train}, the cross entropy loss of sample X under the parameter Θ is $\mathcal{L}(\Theta, X)$, and the model parameter estimate $\hat{\theta}$ are obtained by minimizing the empirical risk:

$$\hat{\theta} = \arg\min_{\theta\in\Theta} \frac{1}{|D_{train}|} \sum_{X\in D_{train}} \mathcal{L}(\Theta, X). \tag{1}$$

The influence function is defined as the derivative of the parameter estimate $\hat{\theta}$ with respect to a small perturbation ϵ applied to some samples D_f within the D_{train}. After perturbation, the parameter estimate is

$$\hat{\theta}_{\epsilon,D_f} = \arg\min_{\theta\in\Theta} \left[\frac{1}{|D_{train}|} \sum_{X\in D_{train}} \mathcal{L}(\Theta, X) + \epsilon \sum_{X^f\in D_f} \mathcal{L}(\Theta, X^f) \right]. \tag{2}$$

The derivative of $\hat{\theta}_{\epsilon,D_f}$ with respect to ϵ can be expressed as

$$\frac{d\hat{\theta}_{\epsilon,D_f}}{d\epsilon}|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \sum_{X^f\in D_f} \nabla_\theta \mathcal{L}(\Theta, X^f), \tag{3}$$

where $H_{\hat{\theta}}^{-1} = \nabla_\theta^2 \mathcal{L}(\Theta, D_{train})^{-1}$. If $\epsilon = -\frac{1}{|D_{train}|}$, we can unlearn samples D_f in D_{train} as mentioned in Eq. (2) and get the change of the parameter θ as

$$\Delta\theta(D_f) = H_{\hat{\theta}}^{-1} \frac{1}{|D_{train}|} \sum_{X^f\in D_f} \nabla_\theta \mathcal{L}(\Theta, X^f). \tag{4}$$

3.2. Backdoor detection based on unlearning perturbation

Unlearning perturbation and impact. When a slight parameter perturbation $\Delta\Theta$ is applied to unlearn normal samples, other samples should show a noticeable change in the model evaluation metric. The model evaluation metrics can include probability prediction, logits output, loss value, etc. In contrast, we expect backdoor samples to change only marginally, because their predictions depend more on the injected trigger than on normal features. This difference in sensitivity between normal and backdoor samples can serve as a reliable signal for backdoor detection. Thus, we propose the concepts of unlearning perturbation and unlearning impact as follows.

Definition 1 (Unlearning perturbation)**.** Considering model parameters Θ trained on D_{train}, the change of model parameters caused by unlearning samples D_f is $\Delta\Theta(D_f)$. If a perturbation $\mu\Delta\Theta(D_f)$ is added to the model parameters, and Θ turn to $\Theta + \mu\Delta\Theta(D_f)$, this process is referred to as an unlearning perturbation applied to the model using D_f:

$$\Theta \leftarrow \Theta + \mu\Delta\Theta(D_f), \tag{5}$$

where $\mu\Delta\Theta(D_f)$ is the value of unlearning perturbation and μ is the perturbation coefficient employed to control the degree of perturbation.

After unlearning perturbation, the model evaluation metrics of other samples are changed. We present the concept of unlearning impact and sensitivity as follows:

Definition 2 (Unlearning impact and sensitivity)**.** Considering model parameters Θ trained on D_{train}, given forgotten samples D_f and a target sample X_c. When an unlearning perturbation applied to the model using D_f, the model evaluation metric F of sample X_c changes, which is termed the unlearning impact of D_f on X_c. The variation in the model evaluation metric for X_c is used to measure the degree of unlearning impact, which is called the unlearning impact value:

$$I(X_c, D_f, \mu, F) = F(\Theta + \mu\Delta\Theta(D_f), X_c) - F(\Theta, X_c), \tag{6}$$

and the unlearning sensitivity of X_c refers to D_f can be defined as

$$S(X_c, D_f, F) = \lim_{\mu\to 0} \frac{I(X_c, D_f, \mu, F)}{\mu}. \tag{7}$$

Unlearning normal samples in backdoor model. For an N-class classification model $M(\Theta)$, the model's logits output of X is $Y = (y_0, y_1, \ldots, y_{N-1})$. Taking the model's logits output corresponding to sample label as the model evaluation metric. To simplify the explanation, the model's logits output corresponding to sample label y_i in the output Y is denoted as $y_i = m_i(\Theta, X)$.

In dirty label backdoor attacks, the category which backdoor samples are inserted is regard as the backdoor attack target category t. Within backdoor attack target category t, we analyze the unlearning impact value of normal samples from test set on the backdoor samples and normal samples in the training set. First, we applied unlearning perturbation on the model using D_f, the value of perturbation is $\mu\Delta\Theta(D_f) = \{\mu\Delta\theta_j(D_f) | \theta_j \in \Theta, j = 1, \ldots, |\Theta|\}$, where $\mu\Delta\theta_j(D_f)$ is calculated by the Eq. (4) from the unlearning algorithm presented in Section 3.1:

$$\begin{aligned} \mu\Delta\theta_j(D_f) &= \mu H_{\theta_j}^{-1} \frac{1}{|D_{train}|} \sum_{X^f\in D_f} \nabla_{\theta_j} \mathcal{L}(\Theta, X^f) \\ &= \mu H_{\theta_j}^{-1} \frac{1}{|D_{train}|} \sum_{X^f\in D_f} \sum_{i\in N} \nabla_{y_i} \mathcal{L}(\Theta, X^f) \nabla_{\theta_j} m_i(\Theta, X^f), \end{aligned} \tag{8}$$

Since the unlearning impact value refers to the model's logits output of the sample category, and in this situation the sample category is t, the other components in unlearning perturbation $\sum_{i\in N} \nabla_{y_i} \mathcal{L}(\Theta, X^f) \nabla_{\theta_j} m_i(\Theta, X^f)$ are omitted when $i \neq t$. In addition, we can consider $\nabla_{y_t} \mathcal{L}(\Theta, X^f)$ as the weight coefficient of each forgotten sample X^f in D_f, and assign the same weight to all samples. When $\mu \to 0$ and $\mu H_{\theta_j}^{-1} \frac{\nabla_{y_t} \mathcal{L}(\Theta, X^f)}{|D_{train}|}$ is infinitesimals of the same order, we can approximate the unlearning perturbation as follows:

$$\begin{aligned} \hat{\mu}\Delta\Theta(D_f) &\approx \hat{\mu}\nabla_\Theta m_t(\Theta, D_f) \\ &= \sum_{X^f\in D_f} \hat{\mu}\nabla_\Theta m_t(\Theta, X^f). \end{aligned} \tag{9}$$

The reason we utilize model's logits output as the metric of unlearning impact rather than probability prediction or loss value is that the softmax function compress the output variance when calculating probability prediction.

Fig. 1 illustrates the distribution of unlearning impact values of normal samples from the test set on samples within the backdoor target category, using unlearning perturbation states in Eq. (9). Evidently, the distribution of unlearning impact value exhibits significant difference between normal samples and backdoor samples. Specifically, the mode of the distribution of unlearning impact values from backdoor samples is closer to zero compared to that of normal samples, indicating that the model's logits output corresponding to sample label of backdoor samples are less susceptible to the unlearning perturbation induced by normal samples in test set.

Analyze unlearning impact via learning dynamics. Consider the perturbation as an update of model parameters and analyze the unlearning impact of samples from the perspective of learning dynamics [24]. Since the magnitude of the perturbation is very small, the first-order Taylor approximation of the unlearning impact value at Θ is

$$\begin{aligned} I(X_c, D_f, \hat{\mu}, m_t) &= m_t(\Theta + \hat{\mu}\Delta\Theta(D_f), X_c) - m_t(\Theta, X_c) \\ &\approx \hat{\mu}\nabla_\Theta m_t(\Theta, X_c) \cdot \Delta\Theta(D_f) \\ &= \hat{\mu}\nabla_\Theta m_t(\Theta, X_c) \cdot \nabla_\Theta m_t(\Theta, D_f), \end{aligned} \tag{10}$$

thus according to Eq. (7) the unlearning sensitivity is

$$\nabla_\Theta m_t(\Theta, X_c) \cdot \nabla_\Theta m_t(\Theta, D_f), \tag{11}$$

which equals to the inner product of the gradient of the model's logits output of the target sample and forgotten samples.

Backdoor detection algorithm. The entire process of backdoor detection is illustrated in the Fig. 2. First, train the backdoor model $M(\Theta)$ on the backdoor dataset D_{train}. For each classification category c, select

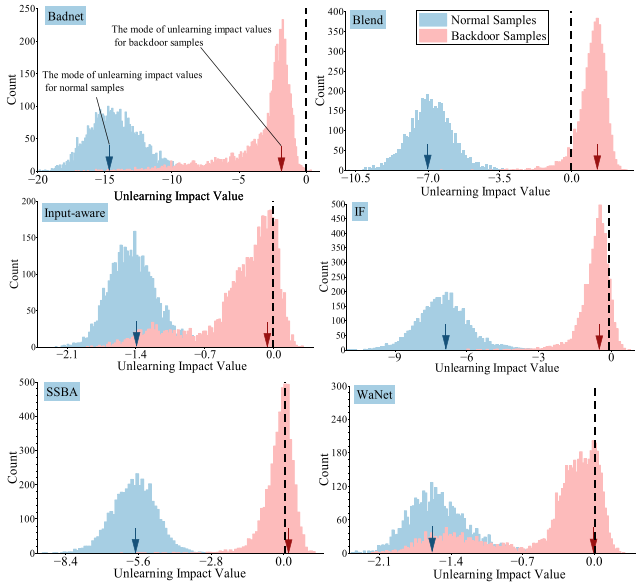


Fig. 1. The distribution of unlearning impact value of samples in the backdoor target category for six backdoor datasets.

Table 1
List of frequently used symbols and their explanations.

M	Trained model.
D_{train}	Training dataset.
D_{test}	Testing dataset.
D_f	The set of forgotten samples.
D_n	The set of normal samples.
D_b	The set of backdoor samples.
Y	Model's logits output, $Y = \{y_0, \dots y_{N-1}\}$.
Θ	Model parameters.
$\mathcal{L}(.)$	The loss function.
t	Backdoor target category.
$p(.)$	The probability prediction for the sample's label.

several normal samples from the test set to construct the forgotten samples D_f^c. Then, perturb the parameters Θ of the backdoor model according to Eq. (9) and calculate unlearning impact values of the samples in category c. If there are no backdoor samples in category c, the unlearning impact values of samples show a concentrated distribution. On the contrary, the unlearning impact values of samples resulting in a bimodal distribution. The distributional differences measured by standard deviation σ_c for each category c and standardize the standard deviation values with the median absolute deviation (MAD):

$$z_c = Z_{MAD}(\sigma_c) = \frac{\sigma_c - \text{Median}_\sigma}{\varsigma \times \text{MAD}}, \tag{12}$$

where Median_σ is the median of the set $\{\sigma_0, \dots, \sigma_{N-1}\}$, MAD is the median of the set $\{|\sigma_i - \text{Median}_\sigma| \mid i = 0, \dots, N-1\}$ and ς usually taken as 1.4826. After that, the category for which standardized standard deviation z_c is greater than τ (the value of τ is determined empirically through experiments) is determined as the backdoor target category.

Within the backdoor target category, a second-order Gaussian mixture model (GMM) is employed to model bimodal distribution and cluster the samples into backdoor samples and normal samples. The cluster of samples whose mode is close to 0 is recognized as backdoor samples. Subsequently, we filter out highly confident backdoor samples, where the posterior probability of belonging to the cluster of these samples is greater than the confidence level ξ (usually close to 1). The complete algorithm for backdoor detection based on unlearning perturbation is shown in Algorithm 1. By using unlearning perturbation and analyzing its impact on the model evaluation metrics of samples, we can detect

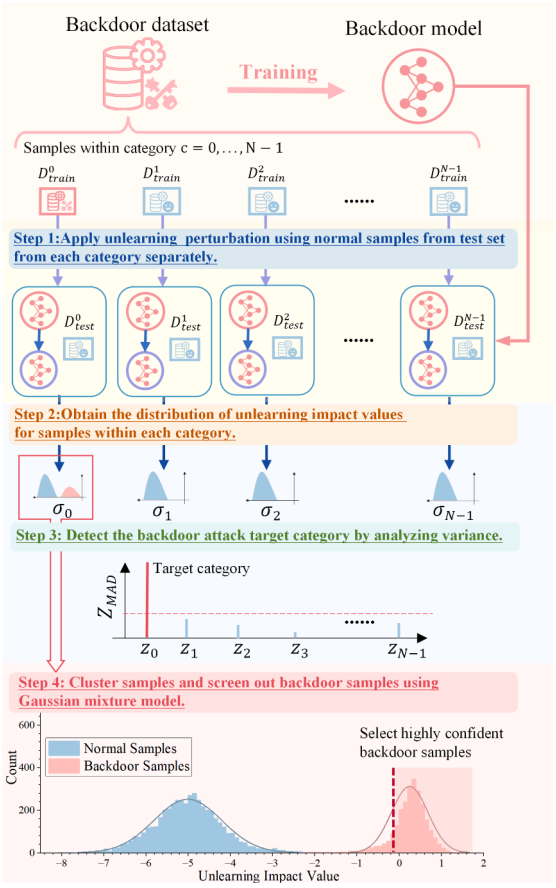


Fig. 2. The process diagram of backdoor detection based on unlearning perturbation and unlearning impact value.

Algorithm 1 Backdoor Detection based on Unlearning Perturbation.

Input: Model $M(\Theta)$, Training set D_{train}, Test set D_{test}, Perturbation coefficient $\hat{\mu}$
Output: Backdoor attack target category t, Backdoor samples D_b

for all category $c = 0, \dots, N-1$ **do**
  $\Upsilon^c_{before} = \{y_i^c | y_i^c = m_c(\Theta, X_i^c), X_i^c \in D^c_{train}\}$ //Calculate the model's logits outputs
  $\tilde{\Theta} = \Theta + \hat{\mu} \sum_{X_i \in D^c_{test}} \nabla_\Theta m_c(\Theta, X_i)$ // Apply unlearning perturbation
  $\Upsilon^c_{after} = \{y_i^c | y_i^c = m_c(\tilde{\Theta}, X_i^c), X_i^c \in D^c_{train}\}$
  $I^c = \{I(X_i^c, D^c_{test}, \hat{\mu}) | X_i^c \in D^c_{train}\}$
  $\sigma_c \leftarrow \sigma(I^c)$ // Calculate the standard deviation
end for
for all category $c = 0, \dots, N-1$ **do**
  $z_c = \frac{\sigma_c - \text{Median}_\sigma}{\varsigma \times \text{MAD}}$ // Standardize the standard deviation with MAD
  if $z_c > \tau$ **then**
    c is target category t of backdoor attack.
  end if
end for
GMM $\leftarrow I^t$
for all samples X_i^t in D^t_{train} **do**
  if $\text{GMM}_{poison}(I(X_i^t, D^t_{test}, \hat{\mu})) > \xi$ **then**
    $D_b \leftarrow D_b \bigcup \{X_i^t\}$
  end if
end for

backdoor samples without additional feature extraction and dimensionality reduction models.

3.3. Backdoor unlearning based on orthogonality-constraint gradient projection

Gradient projection has been widely utilized in continual learning to mitigate the issue of catastrophic forgetting [25]. However, unlike projecting onto the orthogonal space of the sample activation space, our approach involves projecting the gradient into the space that is orthogonal to the gradient space of the main task.

Constrained optimization for backdoor unlearning. After backdoor samples D_b are obtained, a portion of normal samples D_n are selected from each category of the training set or test set. During the process of backdoor unlearning, the goal is to reduce the attack success rate of backdoor samples by fine-tuning model parameters while maintaining the correct recognition of normal samples. The optimization objective is to minimize the empirical risk of unlearning loss function, and the model parameters are updated using the gradient of the loss function $\mathcal{L}_{unlearn}$:

$$\begin{aligned} \min_{\Theta} \quad & \mathcal{L}_{unlearn} = \mathbb{E}_{(X_b,y_b)\sim D_b}\mathcal{L}_{bd}(\Theta, X_b) \\ & \quad +\mathbb{E}_{(X_n,y_n)\sim D_n}\mathcal{L}_{nm}(\Theta, X_n) \\ s.t. \quad & \Theta_{k+1} = \Theta_k - \eta\nabla_\Theta\mathcal{L}_{unlearn}, k = 0, 1, 2, \dots \end{aligned} \tag{13}$$

where

$$\mathcal{L}_{bd}(\Theta, X_b) = -\ln(1 - p(\Theta, X_b))$$
$$\mathcal{L}_{nm}(\Theta, X_n) = -\ln(p(\Theta, X_n)).$$

Here, η represents the learning rate, $p(\cdot)$ represents the probability prediction for the sample's label, X_b and X_n denote a single backdoor sample and normal sample respectively. $\mathcal{L}_{nm}$ is cross-entropy loss function for classification problems, when the probability that model predicts a normal sample as its label increases, the value of $\mathcal{L}_{nm}$ decrease. On the contrary, the loss $\mathcal{L}_{bd}$ aims to reduce the attack success rate of backdoor samples D_b. In detail, if the backdoor samples are classified as the backdoor target category t, the value of $\mathcal{L}_{bd}$ will increase.

To further mitigate the impact of backdoor unlearning on normal samples, inspired by the unlearning impact described in Section 3.2, we propose to eliminate the unlearning impact using orthogonality-constraint gradient projection. Specifically, the loss of the normal samples $\mathcal{L}_{nm}$ can be used to represent the performance of the model for the normal samples. Thus, we use $\mathcal{L}_{nm}$ as the model evaluation metric to measure the unlearning impact value, and unlearning perturbation $\mu\Delta\Theta = -\eta\nabla_\Theta\mathcal{L}_{unlearn}$. The unlearning impact value of the normal samples is

$$\begin{aligned} I(D_n, D_b, \eta, \mathcal{L}_{nm}) &= \mathcal{L}_{nm}(\Theta + \mu\Delta\Theta, D_n) - \mathcal{L}_{nm}(\Theta, D_n) \\ &\approx -\eta\nabla_\Theta\mathcal{L}_{nm}(D_n)\cdot\nabla_\Theta\mathcal{L}_{unlearn}, \end{aligned} \tag{14}$$

which is determined by the inner product of the gradients of the loss functions with respect to the parameters. Based on the definition in Eq. (7), the unlearning sensitivity of a normal sample equals $\nabla_\Theta\mathcal{L}_{nm}(D_n)\cdot\nabla_\Theta\mathcal{L}_{unlearn}$. Projecting $\nabla_\Theta\mathcal{L}_{unlearn}$ onto the null space of $G(\nabla_\Theta\mathcal{L}_{nm})$ forces this inner product to zero, and hence drives the sensitivity to zero. In other words, orthogonality-constraint gradient projection eliminates the unlearning impact on normal samples.

The gradient space J of the main task is spanned by the row vectors in the gradient matrix $G(\nabla_\Theta\mathcal{L}_{nm}(D_n))$ of the normal samples D_n. If the gradient of the unlearning loss $\nabla_\Theta\mathcal{L}_{unlearn}$ can be projected into the space Null(J) that is orthogonal to J, then the unlearning impact value $I(D_n, D_b, \eta, \mathcal{L}_{nm}) \approx 0$; thus, unlearning the backdoor samples X_b in this null space will not affect the classification result of normal samples. The schematic diagram of the gradient projection unlearning method is shown in Fig. 3.

Before the backdoor unlearning, the model parameters are Θ'. After taking into account that the gradient of backdoor unlearning task is in the null space of gradient matrix $G(\nabla_\Theta\mathcal{L}_{nm}(\Theta', D_n))$, the optimization problem of unlearning backdoor samples is

$$\begin{aligned} \min_{\Theta} \quad & \mathcal{L}_{unlearn} = \mathbb{E}_{(X_b,y_b)\sim D_b}\mathcal{L}_{bd}(\Theta, X_b) \\ & \quad +\mathbb{E}_{(X_n,y_n)\sim D_n}\mathcal{L}_{nm}(\Theta, X_n) \\ s.t. \quad & G(\nabla_\Theta\mathcal{L}_{nm}(\Theta', D_n))\cdot\text{Proj}(\nabla_\Theta\mathcal{L}_{unlearn})^T = 0 \\ & \Theta_{k+1} = \Theta_k - \eta\text{Proj}(\nabla_\Theta\mathcal{L}_{unlearn}), k = 0, 1, 2, \dots \end{aligned} \tag{15}$$

In the following part, we need to obtain the basis of the row vectors in gradient matrix $G(\nabla_\Theta\mathcal{L}_{nm}(\Theta', D_n))$, and construct a projector Proj(.) that projects gradients $\nabla_\Theta\mathcal{L}_{unlearn}$ onto the null space of $G(\nabla_\Theta\mathcal{L}_{nm}(\Theta', D_n))$ during backdoor unlearning.

Projector construction. In neural network models, parameters are represented by tensors. Since the number of parameters in neural network is relatively large, we perform gradient projection on parameters individually. For parameter tensors with dimensions greater than 1, such as convolutional kernel parameters and fully connected weights, we perform parameter projection. As for 1-dimensional parameter tensors, identity projector is employed. The projectors $\text{Proj}_\rho(\cdot)$ for different parameter tensors collectively form the projector Proj$(\cdot)$ for the entire gradient of unlearning loss function.

Firstly, acquire a set of normal samples D_n and partition it into S batches B^s, where each batch comprises $|D_n|/S$ samples. Subsequently, calculate the loss $\mathcal{L}_{nm}(B^s)$ for each batch and compute the derivative with respect to the ρth parameter tensor Θ_ρ to obtain the gradient matrix $G_\rho^s \in \mathbb{R}^{m\times n}$ composed of the gradients. For instance, in the convolutional layer, the ρth parameter tensor as the convolutional kernel is $K_\rho \in \mathbb{R}^{C_{out}\times C_{in}\times k\times k}$. After calculating and reshaping operations, the gradient matrix derived from batch s is $G_\rho^s = \text{reshape}(\nabla_{K_\rho}\mathcal{L}_{nm}(B^s)) \in \mathbb{R}^{C_{out}\times C_{in}k^2}$.

Vertically stack the gradient matrices G_ρ^s derived from each batch to obtain a gradient matrix $G_\rho \in \mathbb{R}^{Sm\times n}$. Subsequently, apply singular value decomposition (SVD) to determine the bases of the row vectors within the matrix G_ρ as follows:

$$G_\rho = \begin{bmatrix} G_\rho^1 \\ G_\rho^2 \\ \vdots \\ G_\rho^S \end{bmatrix} = \begin{bmatrix} \widetilde{U} & \overline{U} \end{bmatrix} \begin{bmatrix} \Sigma & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \widetilde{V} & \overline{V} \end{bmatrix}^T, \tag{16}$$

where $U = [\widetilde{U}, \overline{U}] \in \mathbb{R}^{Sm\times Sm}$, $V = [\widetilde{V}, \overline{V}] \in \mathbb{R}^{n\times n}$ and $\Sigma \in \mathbb{R}^{r\times r}$. Here $\widetilde{V}$ are the bases of the row vectors in G_ρ. We choose $I - \widetilde{V}\widetilde{V}^T$ as the projection matrix of the null space of G_ρ, where I represents the identity matrix.

Nevertheless, when constructing the projection matrix, we implemented several strategies to loosen the gradient orthogonality constraint. This also explains the rationale behind incorporating the loss $\mathcal{L}_{nm}(\Theta, D_n)$ into the $\mathcal{L}_{unlearn}$. First, we select the top-q important basis vectors in $\widetilde{V}$ to form $\widehat{V} = \widetilde{V}[:, 0:q] \in \mathbb{R}^{n\times q}$ according to the following criteria:

$$||G_\rho^q||_F^2 \geq \beta\cdot||G_\rho||_F^2, \tag{17}$$

where G_ρ^q represents the matrix reconstructed by top-q important basis, $||\cdot||_F^2$ represents the squared Frobenius norm of the matrix, and β represents compression ratio (defaults to 0.97) . Furthermore, we scale the ith basis vector according to the singular values, as described in the reference [6]. The scaling factor is computed as follows:

$$\lambda_i = \frac{(\alpha+1)\sigma_i}{\alpha\sigma_i + \max(\Sigma)}, \tag{18}$$

here, σ_i denotes the ith singular value of G_ρ, and relaxation factor α is a non-negative scale coefficient hyperparameter that defaults to 10, and $\max(\Sigma)$ represents the maximum value among the diagonal elements of the matrix Σ, where these diagonal elements are the singular values. Finally, the gradient projector Proj_ρ of ρth parameter tensor onto the

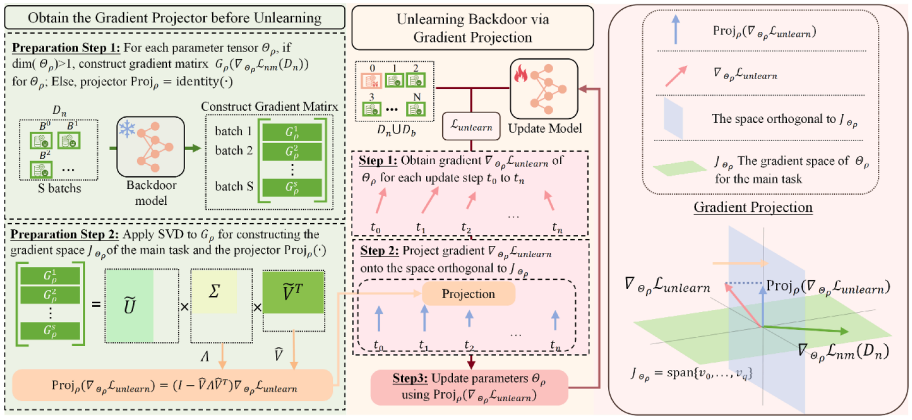


Fig. 3. Schematic diagram of gradient projection method: projecting the gradient of unlearning loss function onto the space orthogonal to the gradient space of the main task.

unlearning gradient $\nabla_{\Theta_\rho}\mathcal{L}_{unlearn}$ with basis scaling is

$$\text{Proj}_\rho(\nabla_{\Theta_\rho}\mathcal{L}_{unlearn}) = (I - \hat{V}\Lambda\hat{V}^T)\nabla_{\Theta_\rho}\mathcal{L}_{unlearn}, \tag{19}$$

where $\Lambda = \text{diag}\{\lambda_0, \ldots, \lambda_{q-1}\}$.

Algorithm 2 Backdoor Unlearning Algorithm via Gradient Projection.

Input: Backdoor model $M(\Theta')$, High-confidence backdoor sample set D_b, Normal sample set D_n, Unlearning epoch N_{epoch}, Relaxation factor α.
Output: Normal model M_{normal}.

for each parameter tensor Θ_ρ in the model **do**
  if dimensions of Θ_ρ greater than 1 **then**
    Use normal samples D_n to calculate the gradient matrix $G(\nabla_{\Theta_\rho}\mathcal{L}_{nm}(\Theta', D_n))$ and applied SVD.
    Construct the projector Proj$_\rho(\cdot)$ as Equation (19).
  else
    Proj$_\rho(\cdot)$ = Identity(·)
  end if
end for
//Unlearning backdoor samples
for $epoch = 0, \ldots, N_{epoch} - 1$ **do**
  for all Batch B in $D_b \bigcup D_n$ **do**
    Calculating the unlearning loss $\mathcal{L}_{unlearn}(B, \Theta)$.
    Using gradient projector Proj$_\rho(\cdot)$ to update each parameter tensor Θ_ρ.
    $\Theta_\rho \leftarrow \Theta_\rho - \eta\text{Proj}_\rho(\nabla_{\Theta_\rho}\mathcal{L}_{unlearn})$
  end for
end for
Output the normal model M_{normal}.

Backdoor unlearning algorithm. In conclusion, our backdoor unlearning method has the following steps. First, after obtaining the backdoor samples D_b, we utilize the normal samples D_n from the training set to calculate the gradient matrix $G(\nabla\mathcal{L}_{nm}(\Theta', D_n))$. Next, according to Eq. (19), construct the projector Proj$_\rho$ for each parameter tensor. Finally, when optimizing the model according to the backdoor unlearning objective, we use the gradient projector to project the gradients. This approach effectively prevents the unlearning process from impairing the model's performance in recognizing normal samples. The complete algorithm process of the backdoor unlearning method based on gradient projection is presented in Algorithm 2.

3.4. Computational complexity analysis

The computational complexity of both the forward propagation and the backward propagation of the model is approximately equivalent, and it is denoted as Q. In the process of backdoor detection, the primary computational burden stems from computing the unlearning impact values of samples in the training set via the forward propagation of the network twice , and also calculating the unlearning perturbation of the normal test dataset once. Consequently, the computational complexity of the backdoor detection algorithm is expressed as $O(2(|D_{train}| + |D_{test}|)Q)$.

During the process of backdoor unlearning, the computational load is attributed to calculating the gradient matrix of the normal samples D_n once, and carrying out 5 times forward and backward propagations respectively on the backdoor samples D_b and the normal samples D_n. Since we only used a portion of the normal samples and backdoor samples in the dataset for backdoor unlearning, it further reduces the computational complexity. Consequently, the computational complexity of the backdoor unlearning is represented as $O((Epoch \times (|D_b| + |D_n|) + 2|D_n|)Q)$.

4. Experiment

4.1. Experimental setup

Attack setting: We evaluate the effectiveness of proposed defense method against six well-known backdoor attacks on the CIFAR-10 and CIFAR-100 dataset, employing PreAct-ResNet18 [26] and ConvNeXt [27] as the neural network architecture. To ensure the comparability of the results, we adopted the mainstream backdoor attack methods listed in BackdoorBench [28], include BadNets, Blended, input-aware dynamic backdoor attack (Input-aware), sample-specific backdoor attack (SSBA), smooth low frequency trigger attack (LF), and warping-based poisoned networks (WaNet). All the backdoor attacks were configured in accordance with the standardized settings provided by BackdoorBench with poisoning rates set at 10% and 5%.

Defense setting: Six backdoor defense methods are compared with respect to their effectiveness against the attack approach. These methods include adversarial neuron pruning (ANP) [29], anti-backdoor learning (ABL) [30], decoupling-based backdoor defense (DBD) [31], reconstructive neuron pruning (RNP) [4], neural polarizer defense (NPD) [5], and maximum margin backdoor mitigation (MM-BD) [3]. All defense configurations and training hyperparameters were calibrated according to the BackdoorBench framework [28] or its official implementation. When performing backdoor defense, to prevent the leakage of samples in the test set, 5% of the normal samples in the training set are utilized. This is the same setting as other defense methods that require normal samples. In addition, the unlearning learning rate is 0.05, and the number of unlearning epoch is 5, which is much less than other methods.

Backdoor defense metrics: We assembled a normal test set consisting of ten thousand normal samples. Moreover, we inserted backdoor into every sample in the test set, except for those in the backdoor target

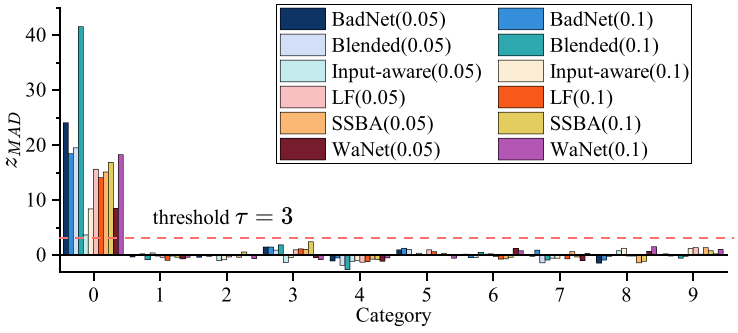


Fig. 4. The z_{MAD} score of each category in the 12 types of CIFAR-10 backdoor datasets, and the value in the brackets represents the poisoning rate.

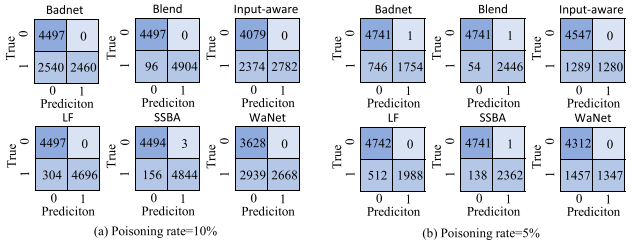


Fig. 5. The confusion matrices for backdoor sample detection in 6 types of CIFAR-10 backdoor datasets, the elements are the number of instances. (a) when the poisoning rate is 10%; (b) when the poisoning rate is 5%.

category t, to construct a backdoor test set. We evaluate the performance of the defense methods using four metrics. Normal accuracy (C-Acc) measures the proportion of normal samples that are correctly classified within the normal test set. The attack success rate (ASR) quantifies the fraction of backdoor samples that are misclassified to the backdoor target label within backdoor test set. Robust accuracy (R-Acc) evaluates the classification accuracy of backdoor samples within backdoor test set to the original categories. The defense effectiveness rating (DER) [32] combines the values of C-Acc and ASR according to the following formula:

$$\mathrm{DER} = [\max(0, \Delta\mathrm{ASR}) - \max(0, \Delta\text{C-Acc}) + 1]/2, \tag{20}$$

where ΔASR denotes the reduction in ASR and ΔC-Acc denotes the decrease in C-Acc after the implementation of backdoor defense. Higher values of C-Acc and R-Acc, lower values of ASR, and higher values of DER indicate superior defense performance. Collectively, these metrics enable a multidimensional assessment of the robustness and effectiveness of the defense methods.

4.2. Backdoor detection results

Firstly, analyze the detection results of the backdoor target category and backdoor samples following the application of the simplified unlearning perturbation. In the 12 different types of the CIFAR-10 backdoor dataset, we derive the result by perturbing the PreAct-ResNet18 model using 128 normal samples from each category and setting $\hat{\mu} = 0.001$. After perturbation, the z_{MAD} score for each category within each dataset is shown in the Fig. 4. From the results, it can be observed that the z_{MAD} of the backdoor target category is significantly higher than that of other categories, and we select $\tau = 3$ as the threshold.

After determining the backdoor target category, we use GMM to filter out the backdoor samples with a posterior probability greater than 99.99%. The confusion matrix of normal samples and backdoor samples is shown in the Fig. 5. In the confusion matrix, the labels of columns correspond to the true classes (0 represents normal samples, and 1 represents backdoor samples) of the data, and the labels of rows correspond to the classes predicted by the GMM. From the results, it can be observed that the detection performance for the Blend and SSBA attacks is relatively well. Since backdoor unlearning can be achieved with only a portion of backdoor samples, the purpose of backdoor detection is to screen out a portion of backdoor samples with high confidence level. It is found that among the samples predicted as backdoor samples, the maximum number of true normal samples is 3.

4.3. Backdoor elimination performance

We trained PreAct-ResNet18 model on six types of CIFAR-10 backdoor dataset, and Tables 2 and 3 present the defense results of UPGP method and other defense methods, with the backdoor poisoning rates being 10% and 5% respectively. When the backdoor poisoning rate is 10%, our method can reduce the average attack success rate of six attacks to 0.34%. In comparison with the RNP method, which is also effective in defending against six types of attacks on the model, our method not only further decreases the average ASR by 0.16 percentage points but also increases the average C-Acc by 1.84 percentage points and the average R-Acc by 2.79 percentage points. When the backdoor poisoning rate is 5%, compared with the case where the poisoning rate is 10%, the average ASR of RNP method increases by 0.47 percentage points, while our method only increases by 0.08 percentage points. This suggests that our method maintains its defense capabilities at a poisoning rate of 5%. In terms of the R-Acc index, our method exhibits strong robustness against six attack methods. The average value of R-Acc index can reach 86.83% and 86.89% when the data poisoning rates are set at 10% and 5% respectively. In conclusion, our method has significant advantages over other defense methods with respect to the indicators of C-Acc, R-Acc, and DER.

Table 4 presents the defense results of the ConvNeXt model trained on the CIFAR-100 backdoor dataset, comparing the performance of ANP, NC [33], and the UPGP method against two typical backdoor attacks, SSBA and WaNet. During the backdoor detection process, the number of normal samples we used for each category is 16, the threshold of target category $\tau = 2.5$, and the confidence level of backdoor samples $\xi = 0.99$. The experimental results demonstrate that the UPGP method can effectively defend against the SSBA and WaNet backdoor attacks. Specifically, it reduces the success rates of the backdoor attacks to 0% and 0.49% respectively. At the same time, this result also indicates that UPGP method also shows effectiveness in datasets with a larger number of categories and in novel convolutional neural networks.

4.4. Ablation study of gradient projection

To analyze the effectiveness of the gradient projection, the backdoor unlearning with and without applying gradient projection is compared under identical settings for PreAct-ResNet18 model trained on CIFAR-10 backdoor dataset. Table 5 illustrates that at poisoning rates of 10% and 5%, the values of ASR have a slight decrease. Meanwhile, the values of C-Acc increase on average by 3.79 percentage points and 2.71 percentage points, respectively, and the values of R-Acc increase on average by 1.32 percentage points and 1.71 percentage points, respectively. This suggests that the gradient projection method can effectively mitigate the impact of backdoor unlearning on the model's ability to recognize normal samples and enhance the defense ability against backdoor samples.

4.5. Impact of the number of normal samples on backdoor unlearning

When conducting backdoor unlearning for PreAct-ResNet18 model trained on CIFAR-10, a number of normal samples are required to construct the projection matrix and restore the model. For further analysis, we tested backdoor unlearning separately under the conditions in which the proportion of normal samples from the training set was

Table 2
Backdoor defense results of PreAct-ResNet18 model trained on CIFAR-10 backdoor dataset at a poisoning rate of 10%. The bold font represents the best result.

Defense →	No Defense				ANP [29]				ABL [30]				DBD [31]			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
BadNets	91.32	95.03	4.67	-	84.17	0.73	84.04	93.58	83.32	**0.00**	89.02	93.52	89.65	1.28	89.17	96.04
Blended	93.47	99.92	0.08	-	86.58	2.19	61.17	95.42	77.30	0.73	64.52	91.51	69.91	99.98	0.02	38.22
LF	93.19	99.28	0.71	-	85.15	0.60	82.72	95.32	71.84	0.00	58.52	88.97	61.71	0.00	54.01	83.90
SSBA	92.88	97.86	1.99	-	86.54	0.39	79.87	95.57	80.79	0.00	84.66	92.89	63.50	99.51	0.39	35.31
Input-aware	90.67	98.26	1.66	-	90.36	1.64	86.99	98.16	63.19	90.16	5.77	40.31	78.60	8.54	75.28	88.83
WaNet	91.25	89.73	9.76	-	85.73	0.43	86.50	91.89	83.19	**0.00**	85.86	90.84	80.90	6.61	77.61	86.39
Average →	92.13	96.68	3.15	-	86.42	1.00	80.22	94.99	76.61	15.15	64.73	83.00	74.05	35.99	49.41	71.45
Defense →	RNP [4]				NPD [5]				MM-BD [3]				Our Method			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
BadNets	88.74	0.90	88.88	95.78	88.51	0.99	84.42	95.62	89.56	2.27	88.97	95.50	**91.30**	1.01	**91.22**	**97.00**
Blended	89.17	0.82	66.80	97.40	91.83	0.19	34.99	99.05	90.11	85.21	12.71	55.68	**92.48**	**0.01**	**69.47**	**99.46**
LF	90.43	0.20	84.91	98.16	90.59	0.89	45.68	97.90	90.60	29.91	64.48	83.39	**92.20**	**0.00**	**89.01**	**99.14**
SSBA	88.54	0.03	79.43	96.74	91.14	0.29	73.34	97.92	89.02	27.73	63.49	83.13	**91.98**	**0.01**	**87.37**	**98.47**
Input-aware	92.75	0.82	91.51	98.72	90.25	**0.19**	82.73	98.83	88.09	3.99	82.49	95.84	**92.64**	0.47	**91.49**	**98.89**
WaNet	92.82	0.22	92.23	94.76	87.35	5.97	82.66	89.93	88.06	3.11	86.20	91.71	**92.91**	0.51	**92.40**	**94.60**
Average →	90.41	0.50	83.96	96.93	89.95	1.42	67.30	96.54	89.24	25.37	66.39	84.21	**92.25**	**0.34**	**86.83**	**97.93**

Table 3
Backdoor defense results of PreAct-ResNet18 model trained on CIFAR-10 backdoor dataset at a poisoning rate of 5%. The bold font represents the best result.

Defense →	No Defense				ANP [29]				ABL [30]				DBD [31]			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
BadNets	92.64	88.74	10.78	-	87.24	0.34	87.34	91.50	84.08	**0.00**	90.50	90.09	87.68	2.11	87.01	90.84
Blended	93.67	99.61	0.39	-	88.00	11.98	58.50	90.98	65.78	0.00	48.48	85.86	75.00	99.99	0.01	40.67
LF	93.35	98.03	1.86	-	86.32	2.66	82.79	94.17	66.54	98.02	1.27	36.60	83.98	15.67	73.11	86.50
SSBA	93.27	94.91	4.82	-	87.53	0.66	81.63	94.26	81.60	0.09	79.76	91.58	72.19	11.34	65.42	81.25
Input-aware	91.52	90.20	8.91	-	91.54	2.22	86.40	93.99	58.14	92.58	4.84	33.31	76.23	6.12	71.99	84.40
WaNet	91.76	85.50	13.49	-	87.21	0.57	87.69	90.19	69.49	95.20	3.79	38.87	72.22	9.93	69.41	78.02
Average →	92.70	92.83	6.71	-	87.97	3.07	80.73	92.51	70.94	47.65	38.11	62.72	77.88	24.19	61.16	76.94
Defense →	RNP [4]				NPD [5]				MM-BD [3]				Our Method			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
BadNets	88.21	0.77	87.92	91.77	89.09	1.31	86.58	91.94	89.33	3.09	88.60	91.17	**92.13**	0.98	**92.00**	**93.63**
Blended	90.71	2.63	62.51	97.01	91.03	0.12	41.94	98.42	90.31	95.01	4.63	50.63	**92.77**	**0.00**	**71.08**	**99.36**
LF	87.46	1.10	80.29	95.52	90.03	0.54	52.93	97.08	90.03	35.19	59.07	79.76	**93.12**	**0.06**	**87.57**	**98.87**
SSBA	88.59	0.04	84.05	95.09	90.45	1.49	65.27	95.30	89.53	14.10	74.58	88.54	**92.42**	**0.02**	**89.89**	**97.02**
Input-aware	92.80	**1.02**	89.23	94.59	90.24	0.29	82.40	94.32	89.64	2.67	83.28	92.83	**92.90**	1.04	**88.98**	**94.57**
WaNet	92.06	0.26	91.89	92.62	90.98	**0.21**	87.58	92.25	89.33	3.88	87.27	89.60	**92.76**	0.44	91.83	**92.53**
Average →	89.97	0.97	82.65	94.43	90.30	0.66	69.45	94.89	89.70	25.66	66.24	82.09	**92.68**	**0.42**	**86.89**	**96.00**

Table 4
Backdoor defense results of ConvNeXt model trained on CIFAR-100 backdoor dataset at a poisoning rate of 5%. The bold font represents the best result.

Defense →	No Defense				ANP			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
SSBA	72.86	90.78	7.58	-	**67.39**	81.95	13.57	51.68
WaNet	63.04	77.66	16.51	-	56.82	15.55	47.44	77.95
Defense →	NC				Our method			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
SSBA	52.13	4.32	45.32	82.86	66.40	**0.00**	**61.18**	**92.11**
WaNet	**66.17**	21.96	51.34	77.85	65.29	**0.49**	**61.68**	**88.59**

Table 5
Backdoor unlearning results of PreAct-ResNet18 model trained on CIFAR-10 with and without gradient projection. The numbers in the brackets represent the performance improvement after using gradient projection.

backdoor = 10%	Without gradient projection			Applied gradient projection			
Attack	C-Acc (%)	ASR (%)	R-Acc (%)	C-Acc (%)		ASR (%)	R-Acc (%)	
BadNets	86.34	1.70	88.26	91.30	(+4.96)	1.01	91.22	(+2.97)
Blended	88.30	0.00	66.70	92.48	(+4.18)	0.01	69.47	(+2.77)
LF	89.45	0.00	88.57	92.20	(+2.75)	0.00	89.01	(+0.44)
SSBA	86.29	0.00	85.68	91.98	(+5.69)	0.01	87.37	(+1.69)
Input-aware	89.69	0.31	91.62	92.64	(+2.95)	0.47	91.49	(−0.13)
WaNet	90.73	0.41	92.21	92.91	(+2.18)	0.51	92.40	(+0.19)
Average	88.47	0.40	85.51	92.25	(+3.79)	0.34	86.83	(+1.32)
backdoor = 5%	Without gradient projection			Applied gradient projection			
Attack	C-Acc (%)	ASR (%)	R-Acc (%)	C-Acc (%)		ASR (%)	R-Acc (%)	
BadNets	88.04	0.99	88.07	92.13	(+4.09)	0.98	92.00	(+3.93)
Blended	90.37	0.01	70.12	92.77	(+2.40)	0.00	71.08	(+0.96)
LF	90.95	0.07	87.33	93.12	(+2.17)	0.06	87.57	(+0.24)
SSBA	87.84	0.16	84.66	92.42	(+4.58)	0.02	89.89	(+5.23)
Input-aware	91.92	0.87	89.28	92.90	(+0.98)	1.04	88.98	(−0.30)
WaNet	90.69	0.47	91.63	92.76	(+2.07)	0.44	91.83	(+0.20)
Average	89.97	0.43	85.18	92.68	(+2.71)	0.42	86.89	(+1.71)

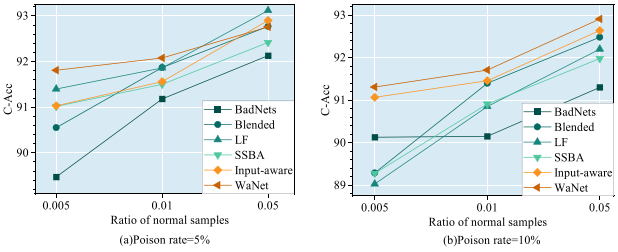


Fig. 6. C-Acc values after backdoor unlearning for PreAct-ResNet18 model trained on CIFAR-10 under different proportion of normal samples in normal dataset.(a) when the poisoning rate is 5%; (b) when the poisoning rate is 10%.

5%, 1%, and 0.5% respectively. As shown in Table 6, alterations in the quantity of normal samples can influence the C-Acc and ASR of the model after backdoor unlearning. It can be observed in Fig. 6 that when the poisoning rate is 10%, as the rate of normal samples increases from 0.5% to 5%, the average C-Acc increases by 2.23 percentage points. However, data imbalance will have an impact on the backdoor unlearning. The loss function for backdoor unlearning is composed of two components: the loss for achieving backdoor elimination and the loss related to maintaining the recognition ability of normal samples. When the number of normal samples decreases, the loss for maintaining the recognition of normal samples also decreases. This results in having a stronger backdoor elimination ability, but simultaneously, the model's recognition capability for normal samples declines.

4.6. The main differences with other methods

The existing methods have provided a great deal of inspiration for our proposed method, but there are also notable distinctions between them.

The ABL method [30] identifies the backdoor samples based on the fact that the loss of backdoor samples decreases more rapidly than that of normal samples in the early stage of training. However, our method analyzes the impacts on samples from the perspective of unlearning samples, and further illustrates from the perspective of learning dynamics that the differences in the model behaviors of backdoor samples and normal samples during the training and unlearning processes are caused by the gradient inner product. In addition, compared to the method that screen backdoor samples from the dynamic learning process, our method, which conducted a single unlearning perturbation, can more rapidly identify a subset of backdoor samples with high confidence. The advantages of the perturbation method also confirm the philosophical idea that destruction is easier than construction.

The ANP [29], RNP [4] methods eliminate the backdoor by identifying the neurons related to the backdoor and then pruning them. Besides, MM-BD [3], NPD [5] methods restrict or change the activation of backdoor samples by introducing additional structures into the model. Rather than analyzing the feature activation, we project the gradient of the unlearning loss function onto the orthogonal space of the main task, thereby mitigating the impact on the main task. Furthermore, the process of calculating the gradient projection matrix has strong theo-

Table 6
The impact of the number of normal samples on backdoor unlearning for PreAct-ResNet18 model trained on CIFAR-10. 'Rate' is the proportion of normal samples from the training dataset.

poisoning rate = 10%	normal rate = 5%				normal rate = 1%				normal rate = 0.5%			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
BadNets	91.30	1.01	91.22	97.00	90.15	0.74	90.73	96.55	90.13	0.41	91.06	96.71
Blended	92.48	0.01	69.47	99.46	91.40	0.02	71.52	98.92	89.29	0.00	68.49	97.87
LF	92.20	0.00	89.01	99.14	90.85	0.00	90.18	98.47	89.03	0.01	90.83	97.55
SSBA	91.98	0.01	87.37	98.47	90.91	0.02	87.82	97.93	89.28	0.02	88.12	97.12
Input-aware	92.64	0.47	91.49	98.89	91.46	0.58	90.86	98.84	91.07	0.54	90.70	98.86
WaNet	92.91	0.51	92.40	94.60	91.71	0.56	91.31	94.58	91.31	0.37	91.41	94.67
Average →	92.25	0.34	86.83	97.93	91.08	0.32	87.07	97.55	90.02	0.23	86.77	97.13
poisoning rate = 5%	normal rate = 5%				normal rate = 1%				normal rate = 0.5%			
Attack ↓	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)	C-Acc(%)	ASR(%)	R-Acc(%)	DER(%)
BadNets	92.13	0.98	92.00	93.63	91.18	0.38	91.79	93.45	89.47	0.19	90.90	92.69
Blended	92.77	0.00	71.08	99.36	91.87	0.02	73.09	98.90	90.55	0.00	71.52	98.25
LF	93.12	0.06	87.57	98.87	91.86	0.09	91.06	98.22	91.40	0.09	91.17	97.99
SSBA	92.42	0.02	89.89	97.02	91.50	0.02	89.76	96.56	91.02	0.02	89.72	96.32
Input-aware	92.90	1.04	88.98	94.57	91.56	1.54	87.43	94.32	91.03	1.24	87.02	94.23
WaNet	92.76	0.44	91.83	92.53	92.08	0.66	91.51	92.42	91.81	0.67	91.38	92.42
Average →	92.68	0.42	86.89	96.00	91.68	0.45	87.44	95.65	90.88	0.37	86.95	95.32

retical support, so the calculation results are more stable than those of stochastic optimization.

5. Conclusion

This study addresses the post-training removal of backdoors while preserving a model's primary task. Unlearning perturbation is introduced to expose the differing sensitivities of clean and backdoor samples, enabling identification of the target class and high-confidence poisoned inputs without auxiliary feature extractors. An orthogonality-constrained gradient projection method then eliminates the detected backdoor samples while preserving the classification accuracy of the model on normal samples. Experiments on six attack types and two network architectures show that the proposed UPGP method reduces the average attack success rate by over 96 percentage points, while maintaining the model's classification accuracy on normal samples. Furthermore, machine unlearning technology is applied to anomaly detection for the first time, thereby enabling effective elimination of backdoor functionality in the model and furnishing a new perspective and technical means for analysing the relationship between backdoor and normal samples. Future work will explore the use of unlearning perturbation method for detecting out-of-distribution samples and for removing model backdoors in more complex scenarios, including multiple attack targets and clean-label settings.

CRediT authorship contribution statement

Jingtai Li: Writing – original draft, Methodology; **Xiujiu Yuan:** Writing – review & editing, Supervision, Funding acquisition; **Jiwei Tian:** Investigation, Formal analysis; **Shiwei Lu:** Visualization, Validation, Resources; **Dengxiu Yu:** Methodology, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work is supported by National Natural Science Foundation of China under Grant 62402520, Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education of China under Grant KLCS20240202, Natural Science Basic Research Program of Shaanxi Province under Grant 2024JC-YBQN-0637

References

[1] W. Tang, J. Li, Y. Rao, Z. Zhou, F. Peng, A trigger-perceivable backdoor attack framework driven by image steganography, Pattern Recognit. 161 (2025) 111262.
[2] M. Goldblum, D. Tsipras, C. Xie, X. Chen, A. Schwarzschild, D. Song, A. Madry, B. Li, T. Goldstein, Dataset security for machine learning: data poisoning, backdoor attacks, and defenses, IEEE Trans. Pattern Anal. Mach. Intell. 45 (2) (2023) 1563–1580.
[3] H. Wang, Z. Xiang, D.J. Miller, G. Kesidis, MM-BD: post-training detection of backdoor attacks with arbitrary backdoor pattern types using a maximum margin statistic, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1994–2012.
[4] Y. Li, X. Lyu, X. Ma, N. Koren, L. Lyu, B. Li, Y.-G. Jiang, Reconstructive neuron pruning for backdoor defense, in: Proceedings of the 40th International Conference on Machine Learning, 2023, pp. 19837–19854.
[5] M. Zhu, S. Wei, H. Zha, B. Wu, Neural polarizer: a lightweight and effective backdoor defense via purifying poisoned features, in: Advances in Neural Information Processing Systems, 2023, pp. 1132–1153.
[6] G. Saha, K. Roy, Continual learning with scaled gradient projection, in: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence, 37, 2023, pp. 9677–9685.
[7] D. Tang, X. Wang, H. Tang, K. Zhang, Demon in the variant: statistical analysis of DNNs for robust backdoor contamination detection, in: 30th USENIX Security Symposium (USENIX Security 21), 2021, pp. 1541–1558.
[8] B. Hu, C.-H. Chang, Diffense: defense against backdoor attacks on deep neural networks with latent diffusion, IEEE J. Emerg. Sel. Top. Circuits Syst. 14 (4) (2024) 729–742.
[9] X. Liu, M. Li, H. Wang, S. Hu, D. Ye, H. Jin, L. Wu, C. Xiao, Detecting backdoors during the inference stage based on corruption robustness consistency, in: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 16363–16372.
[10] C. Chen, H. Hong, T. Xiang, M. Xie, Anti-Backdoor model: a novel algorithm to remove backdoors in a non-invasive way, IEEE Trans. Inf. Forensics Secur. 19 (2024) 7420–7434.
[11] H. Wang, S. Karami, O. Dia, H. Ritter, E. Emamjomeh-Zadeh, J. Chen, Z. Xiang, D.J. Miller, G. Kesidis, Training set cleansing of backdoor poisoning by self-supervised representation learning, in: ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2023, pp. 1–5.
[12] Y. Li, H. Ma, Z. Zhang, Y. Gao, A. Abuadbba, M. Xue, A. Fu, Y. Zheng, S.F. Al-Sarawi, D. Abbott, NTD: non-transferability enabled deep learning backdoor detection, IEEE Trans. Inf. Forensics Secur. 19 (2024) 104–119.
[13] L. Huang, L.Y. Zhang, C.-C. Chang, W. Wang, C. Qin, Backdoor defense based on adversarial prediction proximity and contrastive knowledge distillation, Pattern Recognit. 172 (2026) 112336.

[14] Y. Feng, B. Ma, D. Liu, Y. Zhang, W. Cai, Contrastive neuron pruning for backdoor defense, IEEE Trans. Image Process. 34 (2025) 1234–1245.
[15] Y. Liu, M. Fan, C. Chen, X. Liu, Z. Ma, L. Wang, J. Ma, Backdoor defense with machine unlearning, in: IEEE INFOCOM 2022 - IEEE Conference on Computer Communications, 2022, pp. 280–289.
[16] H. Chen, T. Zhu, L. Zhang, B. Liu, D. Wang, W. Zhou, M. Xue, QUEEN: Query unlearning against model extraction, IEEE Trans. Inf. Forensics Secur. 20 (2025) 2143–2156.
[17] S. Liu, Y. Yao, J. Jia, S. Casper, N. Baracaldo, P. Hase, Y. Yao, C.Y. Liu, X. Xu, H. Li, K.R. Varshney, M. Bansal, S. Koyejo, Y. Liu, Rethinking machine unlearning for large language models, Nat. Mach. Intell. 7 (2) (2025) 181–194.
[18] A.K. Tarun, V.S. Chundawat, M. Mandal, M. Kankanhalli, Fast yet effective machine unlearning, IEEE Trans. Neural Netw. Learn. Syst. 35 (9) (2024) 13046–13055.
[19] L. Bourtoule, V. Chandrasekaran, C.A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot, Machine unlearning, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 141–159.
[20] T. Baumhauer, P. Schottle, M. Zeppelzauer, Machine unlearning: linear filtration for logit-based classifiers, Mach. Learn. 111 (9) (2022) 3203–3226.
[21] W. Wang, Z. Tian, C. Zhang, S. Yu, SCU: an efficient machine unlearning scheme for deep learning enabled semantic communications, IEEE Trans. Inf. Forensics Secur. 20 (2025) 547–558.
[22] Z. Izzo, M. Anne Smart, K. Chaudhuri, J. Zou, Approximate data deletion from machine learning models, in: Proceedings of the 24th International Conference on Artificial Intelligence and Statistics, 2021, pp. 2008–2016.
[23] X.J. Meng, Y. Yang, X.M. Liu, N. Jiang, Active forgetting via influence estimation for neural networks, Int. J. Intell. Syst. 37 (11) (2022) 9080–9107.
[24] S. Guo, Y. Ren, S.V. Albrecht, K. Smith, lpNTK: better generalisation with less data via sample interaction during learning, in: The Twelfth International Conference on Learning Representations, 2024.
[25] C. Yang, M. Dong, X. Zhang, J. Qi, A. Zhou, Introducing common null space of gradients for gradient projection methods in continual learning, in: Proceedings of the 32nd ACM International Conference on Multimedia, 2024, pp. 5489–5497.
[26] K. He, X. Zhang, S. Ren, J. Sun, Identity mappings in deep residual networks, in: Computer Vision–ECCV 2016: 14th European Conference, 2016, pp. 630–645.
[27] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, S. Xie, A ConvNet for the 2020s, in: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 11966–11976.
[28] B. Wu, H. Chen, M. Zhang, Z. Zhu, S. Wei, D. Yuan, C. Shen, BackdoorBench: a comprehensive benchmark of backdoor learning, in: Advances in Neural Information Processing Systems, 35, 2022, pp. 10546–10559.
[29] D. Wu, Y. Wang, Adversarial neuron pruning purifies backdoored deep models, in: Proceedings of the 35th International Conference on Neural Information Processing Systems, 2021, pp. 16913–16925.
[30] Y. Li, X. Lyu, N. Koren, L. Lyu, B. Li, X. Ma, Anti-Backdoor learning: training clean models on poisoned data, in: Advances in Neural Information Processing Systems, 2021, pp. 14900–14912.
[31] K. Huang, Y. Li, B. Wu, Z. Qin, K. Ren, Backdoor defense via decoupling the training process, in: International Conference on Learning Representations, 2022.
[32] M. Zhu, S. Wei, L. Shen, Y. Fan, B. Wu, Enhancing fine-tuning based backdoor defense with sharpness-aware minimization, in: 2023 IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 4443–4454.
[33] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, B.Y. Zhao, Neural cleanse: identifying and mitigating backdoor attacks in neural networks, in: 2019 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2019, pp. 707–723.